Project – High Level Design

On

Marketing Content Tool

Course Name: Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	GAURAV RAUNDHALE	EN22IT301036
2	SANIYA KHAN	EN22IT301091
3	SOHAM SINGH KUSHWAHA	EN22IT301104
4	VEDANT MEENA	EN22IT301120

Group Name:08D12

Project Number: D046

Industry Mentor Name: Mr.Yash

University Mentor Name:Prof.Ajaj Khan

Academic Year: 2026

Table of Contents

1. Introduction.

1.1. Scope of the document.

1.2. Intended Audience

1.3. System overview.

2. System Design.

2.1. Application Design

2.2. Process Flow.

2.3. Information Flow.

2.4. Components Design

2.5. Key Design Considerations

2.6. API Catalogue.

3. Data Design.

3.1. Data Model

3.2. Data Access Mechanism

3.3. Data Retention Policies

3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements

7.1. Security Aspects

7.2. Performance Aspects

8. References

1.Introduction.

1.1Scope of the document-

The **GenAI Marketing Content Tool** is an AI-powered application designed to automate and enhance the process of creating marketing content. In today's digital era, businesses require high-quality, engaging, and consistent content across multiple platforms, which can be time-consuming and resource-intensive.

This project leverages **Generative AI (GenAI)** to generate marketing content such as social media posts, blog ideas, ad copies, and promotional messages based on user inputs. By integrating advanced language models through APIs, the system provides context-aware, creative, and customized content in real time.

The application aims to simplify content creation for marketers, startups, and businesses by reducing manual effort, improving productivity, and ensuring consistent content quality. It offers a user-friendly interface where users can input requirements, select content types, and instantly receive AI-generated results.

Overall, this project demonstrates the practical implementation of AI in digital marketing, showcasing how automation and intelligent systems can transform traditional content creation workflows.

1.2Intended Audience-

The application is designed for individuals and organizations involved in content creation and digital marketing, including:

- **Digital Marketers**
  Professionals who need quick, engaging, and optimized content for campaigns.
- **Small Businesses & Startups**
  Teams with limited resources who want to create high-quality marketing content without hiring large content teams.
- **Content Creators & Social Media Managers**
  Users managing multiple platforms who require consistent and creative posts.
- **Freelancers & Copywriters**
  Individuals looking to speed up their content generation process and improve productivity.
- **Students & Beginners in Marketing**
  Learners who want to explore AI-driven content creation tools for practice and learning.

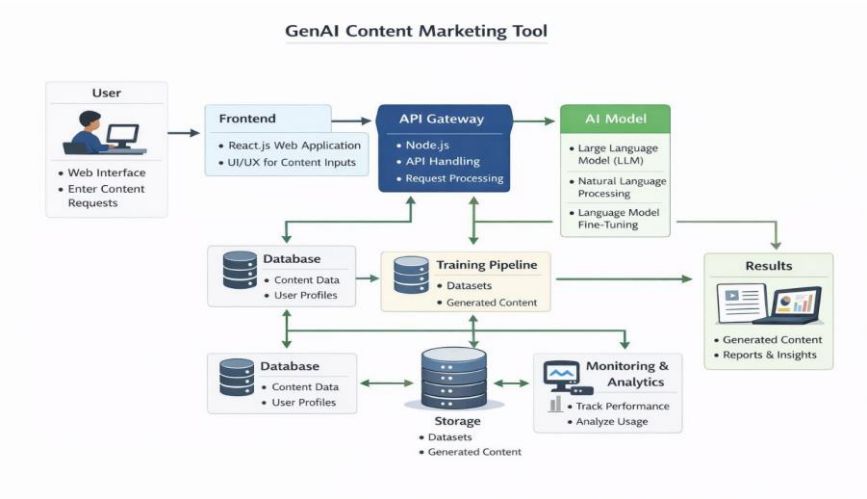
2.System overview-

The **GenAI Marketing Content Tool** is a web-based application that enables users to generate marketing content using Generative AI. The system follows a client-server architecture where the frontend provides an interactive interface, and the backend handles business logic, API integration, and content generation.
Users interact with the system by entering their requirements, such as content type (e.g., social media post, blog idea, ad copy), target audience, and keywords. The backend processes this input and sends a request to a Generative AI API (such as OpenAI/OpenRouter). The AI model then generates relevant and context-aware content, which is returned to the user through the frontend.
The system is designed to be scalable, efficient, and user-friendly. It ensures fast response times and allows users to regenerate or refine content based on their needs. The modular architecture enables easy integration of additional features like content history, templates, and multi-language support in the future.

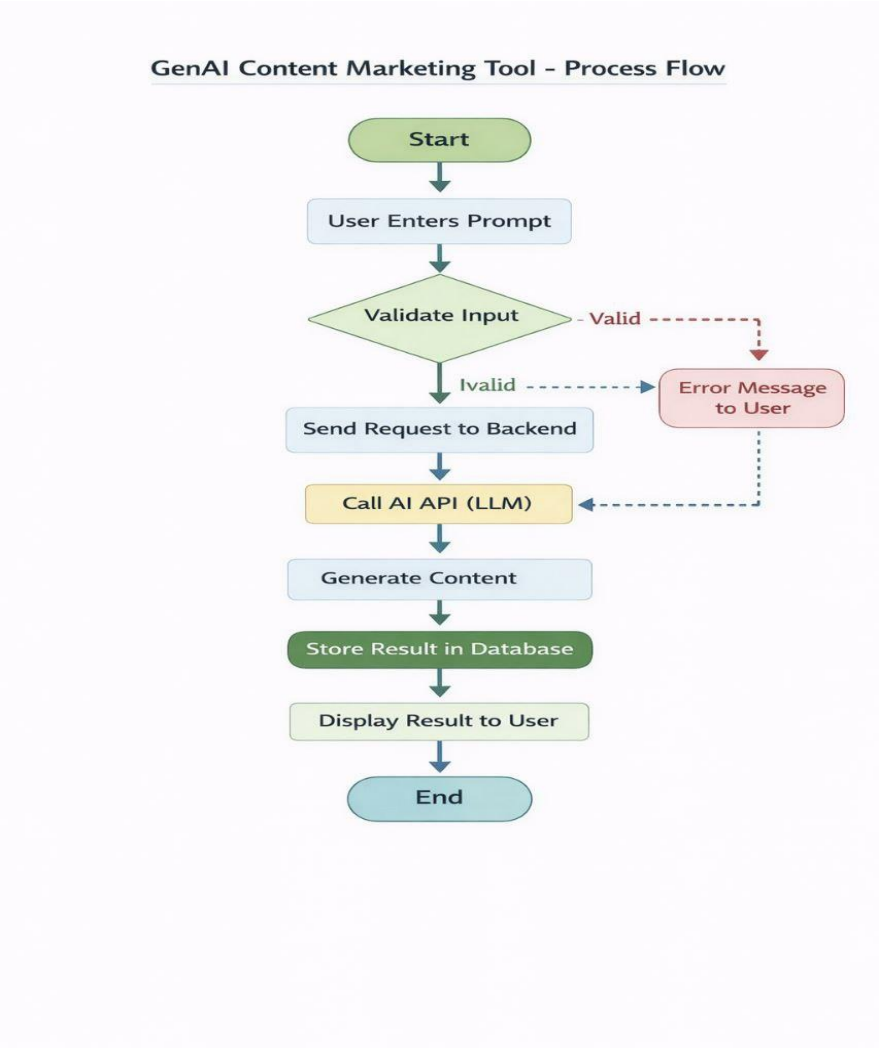
Key Components

- Frontend (UI Layer): Handles user interaction and input
- Backend (Application Layer): Processes logic and API communication
- AI Service (GenAI API): Generates content based on prompts

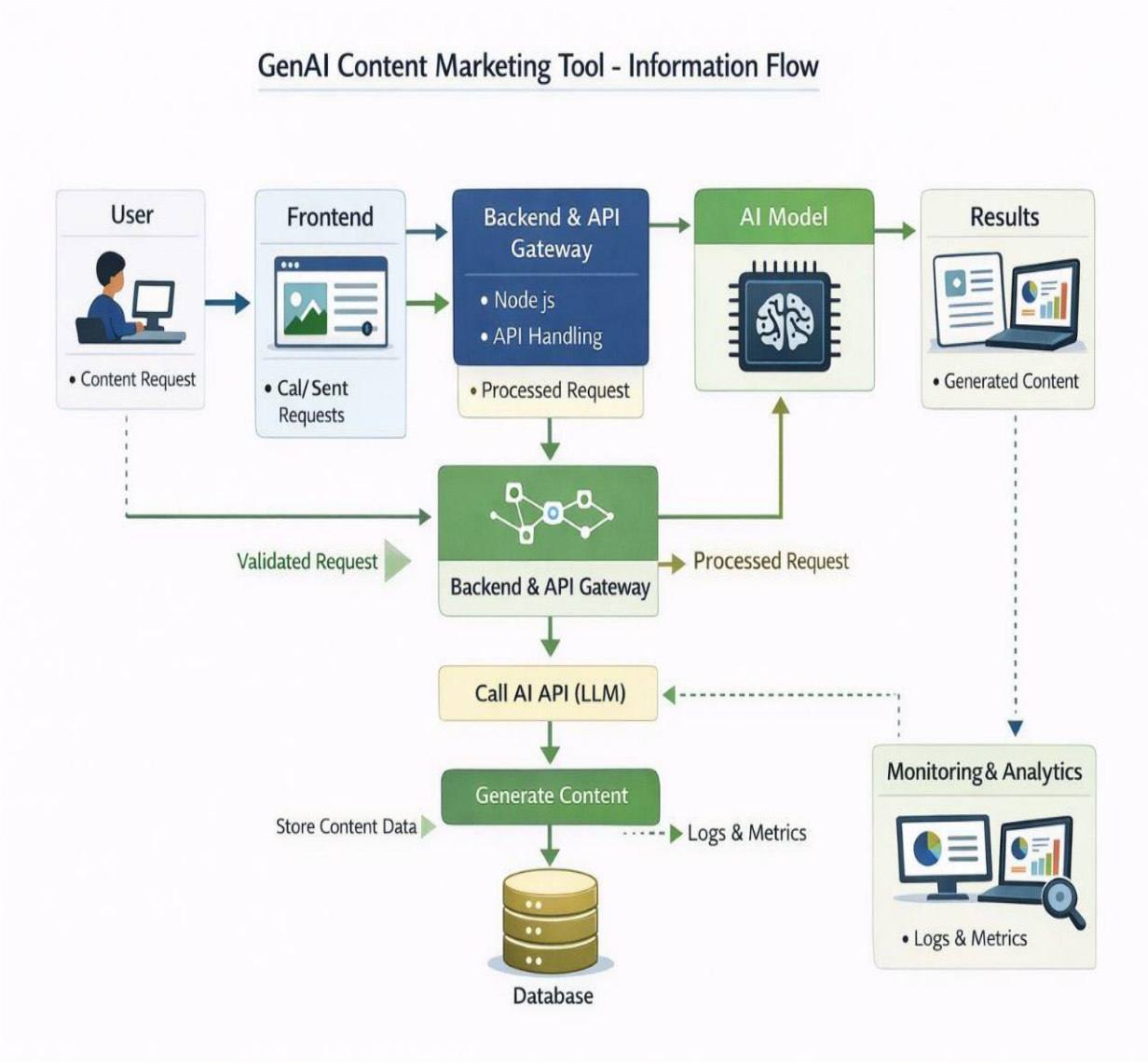
2.1Application Design



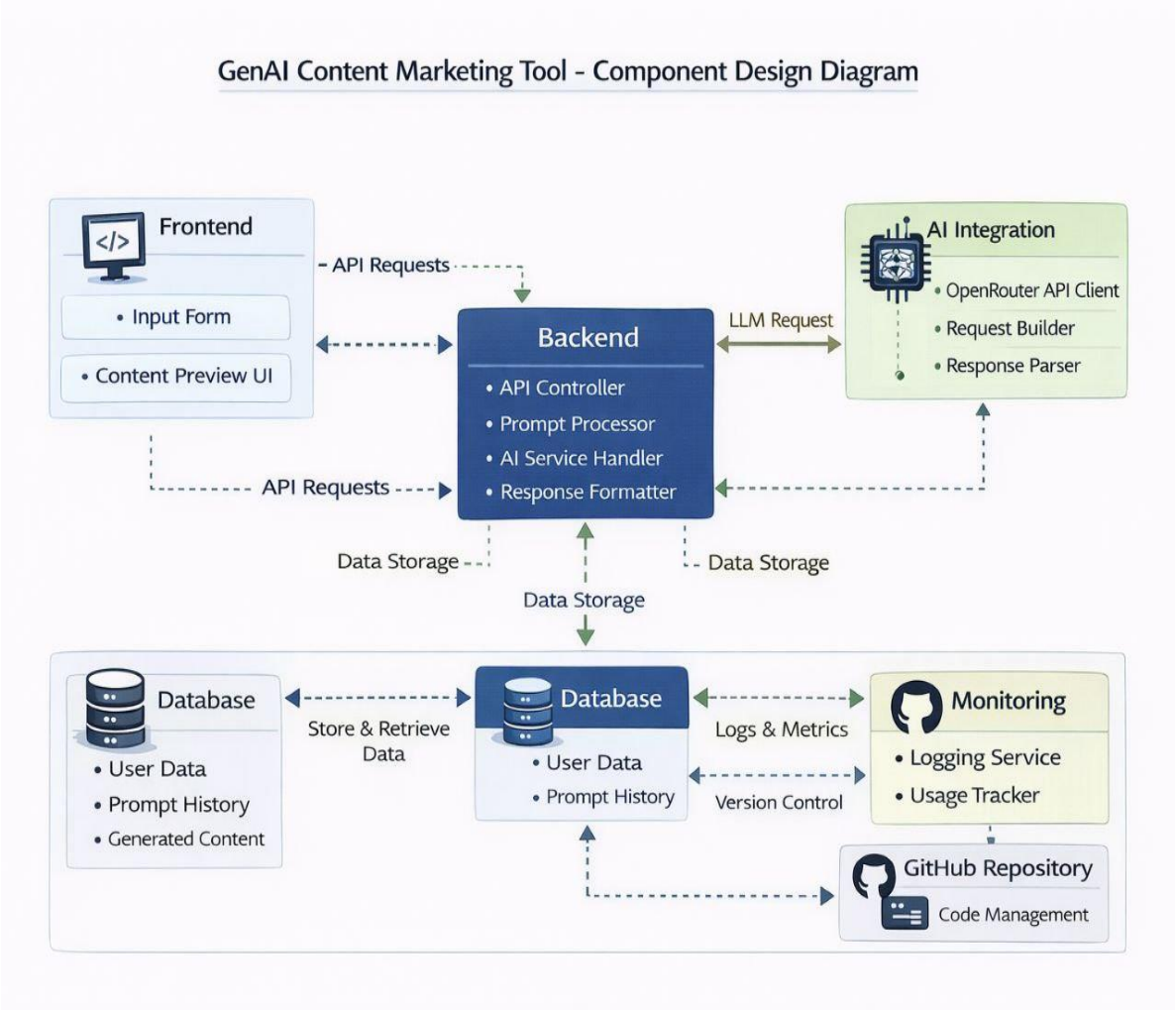
2.2Process Flow-



2.3Information Flow:-



2.4Component Design



2.5. Key Design Considerations

- **Scalability**
  The system should handle multiple users simultaneously and scale efficiently as user demand increases, especially during peak usage.
- **Performance & Response Time**
  Fast content generation is critical. The system should minimize API latency and provide quick responses to enhance user experience.

- **Usability & User Experience (UI/UX)**
  The interface should be simple, intuitive, and user-friendly so that even non-technical users can easily generate content.
- **Reliability & Availability**
  The system should ensure high uptime and handle API failures or network issues gracefully with proper error handling.
- **Security**
  Sensitive data such as API keys and user inputs must be securely managed using environment variables and secure communication protocols (HTTPS).
- **Cost Optimization**
  Since AI APIs are paid, the system should optimize usage by reducing unnecessary API calls and implementing efficient prompt design.
- **Modularity & Maintainability**
  The architecture should be modular, making it easy to update components, fix bugs, or add new features without affecting the entire system.
- **Extensibility**
  The system should allow future enhancements like multi-language support, content templates, tone customization, and analytics.
- **Accuracy & Content Quality**
  Generated content should be relevant, coherent, and aligned with user input to ensure high-quality outputs.

2.6. API Catalogue.

The system uses internal and external APIs to handle user requests and generate AI-based marketing content.

1. Content Generation API

- **Endpoint:** /generate-content
- **Method:** POST
- **Description:** Generates marketing content based on user input

Request Parameters:

- content_type (e.g., social media post, blog, ad copy)
- topic
- target_audience
- tone (optional)

Response:

- generated_content (AI-generated output)
- status

2. Regenerate Content API

- **Endpoint:** /regenerate-content

- **Method:** POST
- **Description:** Generates a new variation of previously generated content

Request Parameters:

- previous_input
- modifications (optional)

Response:

- new_content
- status

3. User Input Validation API

- **Endpoint:** /validate-input
- **Method:** POST
- **Description:** Validates user input before sending to AI API

Response:

- valid/invalid
- error_message (if any)

4. AI Integration API (External)

- **API Used:** OpenAI / OpenRouter
- **Method:** POST
- **Description:** Sends prompt to AI model and receives generated content

Key Parameters:

- model
- prompt
- max_tokens

Response:

- AI-generated text

3.Data Design.

3.1Data Model

The data model represents how data entities are structured and related within the system. It ensures efficient storage, retrieval, and management of user inputs and AI-generated content.

1. Entity Relationship Overview

The system follows a relational data model with three main entities:

- User
- Content Request
- Generated Content

2. Entity Details

User

- Primary entity representing system users
- Each user can generate multiple content requests

Attributes:

- user_id (PK)
- name
- email

Content Request

- Stores user input for content generation

Attributes:

- request_id (PK)
- user_id (FK)
- content_type
- topic
- target_audience
- tone
- created_at

Generated Content

- Stores AI-generated outputs

Attributes:

- content_id (PK)
- request_id (FK)
- generated_text
- version

- created_at

3. Relationships
- User → Content Request
  (1 : Many) → One user can create multiple requests
- Content Request → Generated Content
  (1 : Many) → One request can have multiple generated versions

4. Data Model Type
- Relational Data Model
- Uses Primary Keys and Foreign Keys to maintain relationships
- Ensures data integrity and consistency

5. Why This Data Model?
- Supports content history tracking
- Allows multiple variations (regeneration)
- Keeps system scalable and organized
- Easy to extend (e.g., add templates, analytics)

3.2Data Access Mechanism

The data access mechanism defines how the application interacts with the database and external AI services to store, retrieve, and manage data efficiently.

1. Architecture Approach
The system follows a layered architecture:
- Presentation Layer (Frontend) → Sends user requests
- Application Layer (Backend) → Processes logic
- Data Access Layer (DAL) → Interacts with database
- Database Layer → Stores data

2. Data Access Methods
- Create (Insert)
  Stores new content requests and generated content
- Read (Fetch)
  Retrieves previous content and history
- Update
  Updates user preferences or modifies stored data
- Delete (Optional)
  Removes unwanted or old content

3. API-Based Data Access
   - Backend communicates with:
     - Database using SQL queries / ORM
     - AI API (OpenAI/OpenRouter) via HTTP requests
   - Example flow:
     User Input → Backend → DAL → Database (store request)
     → AI API → Response → DAL → Store generated content

4. Technologies Used
   - ORM (Object Relational Mapping):
     - Example: SQLAlchemy (Python Flask)
     - Simplifies database operations
   - Direct SQL (Optional):
     - For optimized queries
   - REST APIs:
     - For communication between frontend and backend

5. Data Access Optimization
   - Connection Pooling → Efficient DB connections
   - Caching (Optional) → Reduce repeated API calls
   - Indexing → Faster data retrieval

6. Security Considerations
   - Use environment variables for API keys
   - Prevent SQL Injection using ORM / parameterized queries
   - Ensure authentication & authorization (if users exist)

3.3Data Retention Policies

Data retention policies define how long user data and generated content are stored, managed, and deleted to ensure efficiency, privacy, and compliance.

1. Types of Data Stored
   - User Data (Optional): Name, email
   - Content Requests: Input prompts, parameters
   - Generated Content: AI-generated outputs
   - System Logs: Errors and activity logs

2. Retention Duration
   - Content Requests & Generated Content:
     Stored for a limited period (e.g., 30–90 days) for history and reuse
   - User Data:
     Retained until the user deletes their account (if applicable)
   - Logs:
     Stored for short-term monitoring (e.g., 7–14 days)

3. Data Deletion Policy
   - Automatic deletion of old data after retention period
   - Users can manually delete their content/history (optional feature)
   - Soft delete (mark as inactive) or permanent delete based on system design

4. Data Privacy & Security
   - Sensitive data is protected using secure storage practices
   - API keys stored in environment variables
   - Data transmission over HTTPS
   - Access control to prevent unauthorized usage

5. Backup & Recovery
   - Regular backups of important data (if database is used)
   - Ability to restore data in case of system failure

6. Compliance Considerations
   - Follow basic data protection principles (like GDPR concepts)
   - Store only necessary data (data minimization)
   - Ensure transparency in data usage

3.4Data Migration

1. Purpose of Data Migration

   - Move data from local development (SQLite) to production database (MySQL/PostgreSQL)

   - Upgrade database schema without losing existing data

   - Ensure smooth transition during system updates or scaling

2. Migration Scenarios

- Initial Migration:
  Setting up database schema (tables, relationships)

- Schema Migration:
  Adding/modifying fields (e.g., adding tone or version column)

- Data Migration:
  Transferring existing user data and content history to new database

3. Migration Process

1. Backup Existing Data

2. Define Migration Scripts (schema changes)

3. Run Migration Tool (e.g., Flask-Migrate)

4. Validate Data Integrity

5. Deploy Updated System

4. Tools & Technologies

- Flask-Migrate / Alembic → For version-controlled schema changes

- SQL Scripts → For manual migration

- ORM (SQLAlchemy) → For structured database operations

5. Key Considerations

- Data Integrity: No data loss or corruption

- Downtime Minimization: Perform migration with minimal service disruption
- Rollback Strategy: Ability to revert changes if migration fails
- Testing: Validate migration in staging environment before production

6. Migration Strategy

- Use version-controlled migrations
- Apply migrations incrementally
- Maintain backward compatibility when possible

4.Interfaces

1. User Interface (UI)

- Web-based interface for user interaction
- Users can:
  - Enter content requirements (topic, type, audience)
  - Select tone/style
  - View generated content

2. API Interface (Frontend ↔ Backend)

- Communication via **REST APIs**
- Data exchanged in **JSON format**

Example APIs:

- POST /generate-content
- POST /regenerate-content
- GET /content-history

3. Backend ↔ AI Service Interface

- Backend sends requests to **GenAI API (OpenAI/OpenRouter)**
- Uses **HTTP POST requests**

Key Elements:

- Prompt construction
- Model selection
- Response parsing

4. Backend ↔ Database Interface

- Interaction using:
  - **ORM (SQLAlchemy)** or
  - **SQL queries**

Operations:

- Store user inputs
- Save generated content

- Retrieve history

5. External Interfaces

- **AI APIs** → Content generation
- (Optional) **Authentication APIs** → User login/signup

6. Interface Design Considerations

- **Consistency:** Uniform API structure
- **Security:** Authentication & API key protection
- **Performance:** Efficient request-response handling
- **Error Handling:** Proper status codes and messages

5.State and Session Management

1. Stateless Nature of the System

- The backend follows a stateless architecture
- Each request from the frontend contains all necessary information
- No dependency on previous requests

2. Session Management

- A session is used to track a user during their interaction with the application

Approaches:

- Session ID (Cookie-based):
  - A unique session ID is stored in the browser
  - Used to identify user requests
- Token-based (Optional):
  - JWT (JSON Web Token) for authentication
  - Useful for scalable systems

3. State Management

- Minimal state is maintained on the server
- Important data is stored in:
  - Database → content history
  - Frontend (temporary state) → current input/output

4. Handling User Sessions

- Track user activity using session ID or tokens
- Maintain:
    - User preferences (optional)
    - Recent requests

5. Scalability Considerations

- Prefer stateless APIs for horizontal scaling
- Avoid storing large session data on server
- Use token-based authentication for distributed systems

6. Security Considerations

- Secure cookies (HTTPOnly, Secure flags)
- Token expiration (for JWT)
- Prevent session hijacking

6.Caching

Caching is used to store frequently accessed data temporarily to improve performance, reduce latency, and minimize repeated API calls.

1. Purpose of Caching

- Reduce response time
- Minimize expensive AI API calls
- Improve user experience
- Lower operational cost

2. What to Cache

- **Generated Content:**
  Store previously generated results for similar inputs
- **User Requests (Optional):**
  Cache recent queries
- **Static Data:**
  Templates or predefined prompts

3. Caching Strategy

- **Cache-Aside (Lazy Loading):**
  1. Check cache first
  2. If not found → fetch from API/database
  3. Store result in cache

4. Cache Storage Options

- **In-Memory Cache:**
  - Python dictionaries (basic)

  - Redis (recommended for scalability)

- **Browser Cache (Frontend):**
  - Store recent results temporarily

5. Cache Expiry (TTL – Time To Live)

- Cached data is stored for a limited time (e.g., 10–30 minutes)
- Ensures updated and relevant content

6. Cache Invalidation

- Remove cache when:
  - New content is generated
  - User modifies input
  - Cache expires

7.Non-Functional Requirements

7.1 Security Aspects

1. Authentication & Authorization
   - Ensure only authorized users can access the system (if login is implemented)
   - Use JWT (JSON Web Tokens) or session-based authentication
   - Role-based access control (optional for admin/user)

2. API Key Security
   - Store API keys (OpenAI/OpenRouter) in environment variables
   - Never expose keys in frontend or public repositories (e.g., GitHub)
   - Use .env files for secure configuration

3. Data Encryption
   - Use HTTPS for secure data transmission
   - Encrypt sensitive data during communication

4. Input Validation & Sanitization
   - Validate all user inputs before processing
   - Prevent attacks like:
     - SQL Injection
     - Cross-Site Scripting (XSS)

5. Secure Communication
   - All frontend-backend and backend-API communication over secure protocols (HTTPS)
   - Avoid insecure endpoints

6. Rate Limiting & Abuse Prevention
   - Limit number of API requests per user
   - Prevent misuse of AI APIs and reduce cost risks

7. Error Handling & Logging
   - Do not expose sensitive information in error messages
   - Maintain secure logs for debugging and monitoring

8. Data Privacy
   - Store only necessary user data (data minimization)
   - Ensure user data is not shared without consent

9. Protection Against Common Attacks
    - CSRF protection
    - Secure headers (CORS configuration)
    - Session hijacking prevention

7.2 Performance Aspects

1. Response Time
    - The system should generate content within 2–5 seconds (depending on API latency)
    - Fast response is critical for good user experience

2. Throughput
    - The system should handle multiple user requests simultaneously
    - Capable of processing concurrent API calls efficiently

3. Latency Optimization
    - Minimize delay in:
        - Frontend → Backend communication
        - Backend → AI API response

4. Efficient Resource Utilization
    - Optimize CPU and memory usage on the server
    - Avoid unnecessary processing and repeated API calls

5. Caching for Performance
    - Use caching to store frequently generated content
    - Reduces API calls and improves speed

6. Load Handling
    - System should remain stable under increased load
    - Handle peak traffic without crashing

7. Asynchronous Processing (Optional)
    - Use async calls for API requests to avoid blocking
    - Improves responsiveness of the application

8. Database Performance
    - Use indexing for faster data retrieval

- Optimize queries for read/write efficiency

9. Scalability Support
    - Design supports horizontal scaling (adding more servers)
    - Ensures performance does not degrade with growth

8. References

- **OpenAI API Documentation**
https://platform.openai.com/docs
- **OpenRouter API Documentation**
https://openrouter.ai/docs
- **Flask Documentation**
https://flask.palletsprojects.com/
- **SQLAlchemy Documentation**
https://docs.sqlalchemy.org/
- **Flask-Migrate (Alembic) Documentation**
https://flask-migrate.readthedocs.io/
- **REST API Design Guidelines**
https://restfulapi.net/
- **MDN Web Docs (Frontend Development)**
https://developer.mozilla.org/